LEVEL 3 OF 3 · ENTERPRISE

Architecture Guide 3: An enterprise AI platform

Many business units, many assistants and agents, regulated data and real accountability. How to design the shared platform, the governance around it, the repository layout, and the order in which to build it.

USERS 10,000 to 500,000	**TEAMS** Platform team + solution teams	**BUILD TIME** 3 to 12 months, in stages
PATTERN Platform + multi-agent workflows	**RUNS ON** Kubernetes or managed PaaS, multi-region	**CONTROLS** Policy as code, audit, approvals

Nagarjuna Naidu | AI Solution Architect program | October 2026

1. From an application to a platform

At Level 3 the question changes from *how do I build an assistant* to *how do twenty teams build assistants safely without each rebuilding the plumbing*. The **Northwind AI Platform** in this guide offers shared services (models, tools, knowledge, memory, guardrails, approvals, observability), and **solution teams** build assistants and agent workflows on top.

Level 2 (departmental)	Level 3 (enterprise)
One assistant, one team	A platform used by many teams, each with their own solutions
Call the model directly	Every model call goes through a **model gateway**: routing, quotas, caching, logging, fallback
Tools written in the app	Tools published once to a **tool and MCP registry**, with owners, schemas and access policy
Roles in the application	Central identity, policy as code, data classification across all sources
Agent loop in one service	An **agent runtime** for multi-agent workflows with state, checkpoints, approvals and replay
Short-term memory only	Short-term, long-term (episodic, semantic, procedural) with write rules
Evaluation in CI	Evaluation as a shared service, risk-tiered release rings, continuous monitoring
One region	Multi-region, data residency, disaster recovery, SLOs and error budgets
Informal ownership	AI council, model inventory, risk tiers, audit evidence, FinOps

2. The reference architecture

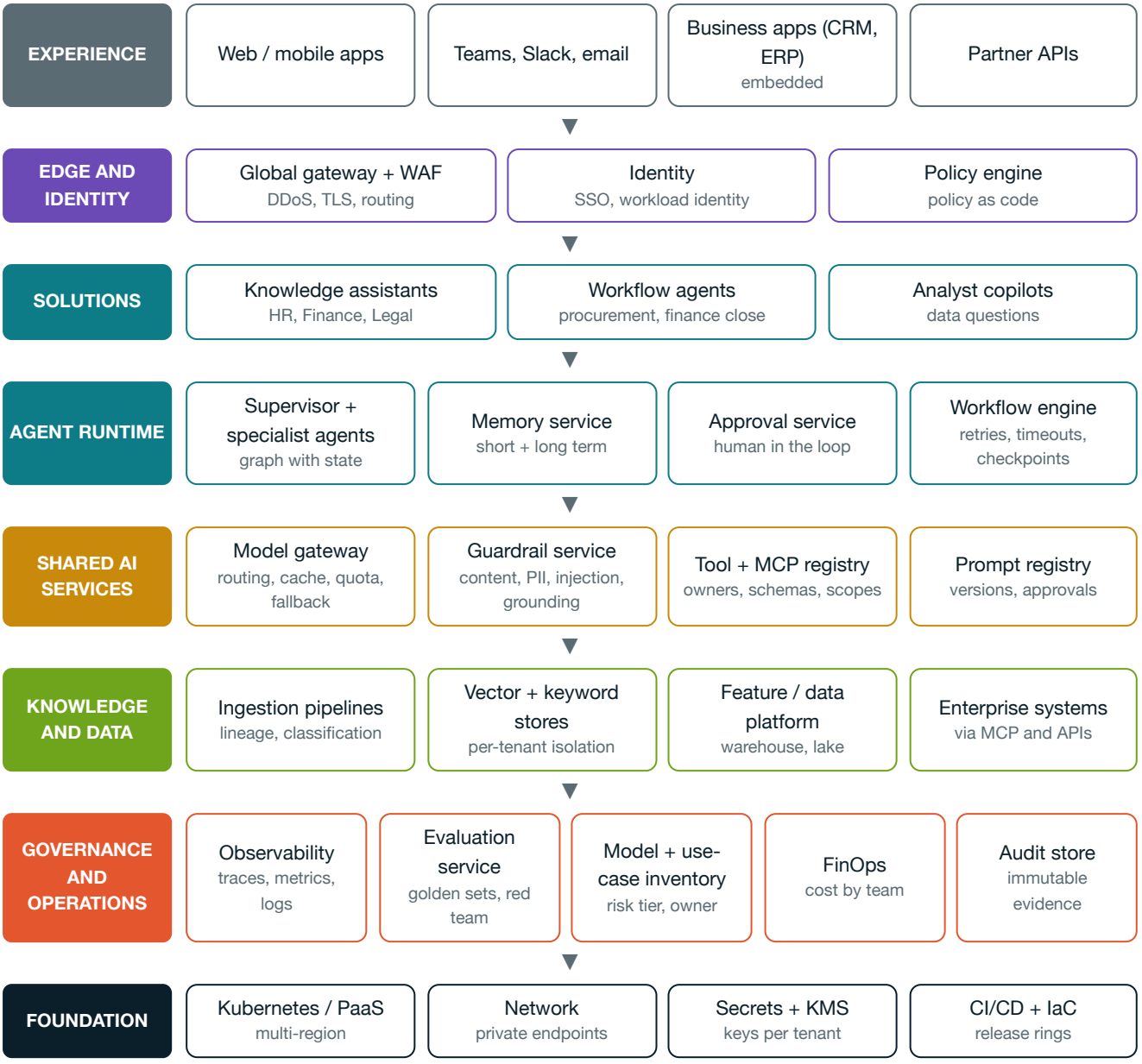


Figure 1. Enterprise reference architecture. Solution teams build in the Solutions row; the platform team owns everything beneath it.

A workflow agent, end to end (procurement example)

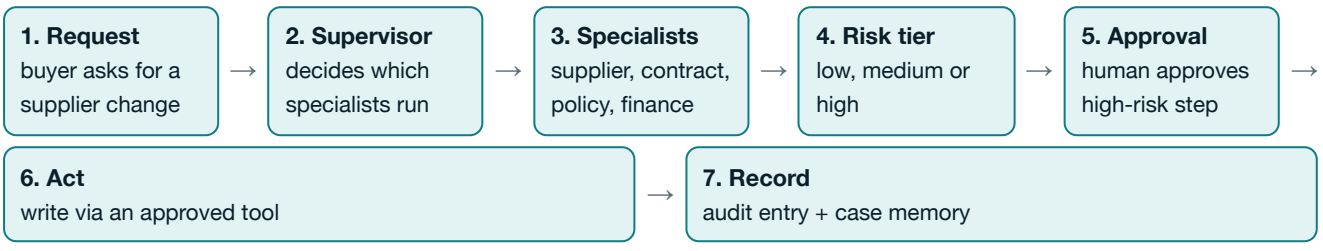


Figure 2. A supervisor coordinates specialist agents; risk decides who must approve; everything is recorded.

3. The platform services, one by one

Service	What it does	Why it exists	Typical choices
Model gateway	One endpoint for all models; routes by task, cost and risk; caches; enforces quotas; falls back when a model is down; logs usage by team	Avoid vendor lock-in, control cost, one place for safety and audit	Azure API Management + a routing service, LiteLLM-style proxy, or a cloud-native gateway
Guardrail service	Checks inputs and outputs: content safety, PII, prompt-injection, grounding against sources, topic limits	Consistent safety without every team re-implementing it	Cloud content-safety service, PII detection libraries, custom rules
Tool and MCP registry	Catalogue of tools with owner, schema, scopes, risk class; MCP servers fronting enterprise systems	Reuse, least privilege, one review process	MCP servers behind a gateway; OpenAPI catalogue
Agent runtime	Runs multi-agent graphs with state, checkpoints, retries, timeouts and human-approval pauses	Long-running, resumable, auditable workflows	LangGraph, Microsoft Agent Framework, a workflow engine such as Temporal
Memory service	Short-term state per run; long-term stores for past cases, facts and playbooks; write rules and expiry	Agents that learn from validated experience without remembering secrets	Redis or Postgres for state; vector store + metadata for long-term; review queue for writes
Knowledge platform	Ingestion with lineage and classification; per-tenant indexes; freshness and deletion	Right data, right people, provable origin	Event-driven pipelines, vector + keyword stores, data catalogue
Approval service	Routes risky steps to the right human with context; records the decision	Human-in-the-loop that is real, not a checkbox	Task queue + chat or ticket integration
Evaluation service	Shared golden sets, red-team suites, regression runs, online sampling	Same quality bar for every team	Evaluation framework + experiment tracking
Observability	End-to-end traces across agents, tools and models; cost and quality dashboards	Find the failing step in minutes	OpenTelemetry, a tracing backend, metrics store
Governance registry	Inventory of models, prompts, tools and use cases with owner and risk tier	Evidence for audits and the AI council	A service backed by a database and an approval workflow

4. Design decisions every enterprise has to make

Decision	Options	How to decide
Single agent or multi-agent	One agent with many tools / supervisor with specialists / peer network	Use multiple agents when roles need **different tools, permissions, prompts or owners**, or when parts can run in parallel. Otherwise one agent is simpler and cheaper
Orchestration pattern	Supervisor, pipeline, router, debate	Supervisor for flexible work; fixed pipeline when the steps never change; router when you only need to pick one specialist

Autonomy level	Suggest only / act after approval / act within limits	Set by **risk tier** of the action, not by how clever the model is
Memory scope	Per run / per user / per team / global	Smallest scope that works; global memory needs review and expiry
Model strategy	One large model / routed small + large / self-hosted for sensitive data	Route by task difficulty and data sensitivity; always have a fallback
Tool access	Direct APIs / MCP servers / a tool gateway	Gateway with scopes per agent; write tools behind approval
Build or buy	Framework, cloud service, product	Buy commodity parts (gateway, safety, tracing); build what differentiates (workflows, data)
Tenancy	Shared index with filters / separate index per unit / separate stack	Isolation proportional to data sensitivity and legal needs

Risk tiers and who approves what

Tier	Examples of actions	Control
0: Read	Search documents, answer questions	Automatic, logged
1: Reversible write	Draft a document, create a ticket, update a note	Automatic with limits; sampled review
2: Business impact	Change a supplier record, issue a small refund	Human approval with the evidence shown
3: Material or irreversible	Large payment, contract change, customer-wide message	Two approvers and a documented reason; never fully autonomous

5. Repository and folder structure

Use a **platform monorepo** for shared services and a **separate repo per solution** (so solution teams release on their own), with shared contracts in a versioned package.

```
northwind-ai-platform/                    (platform monorepo, owned by the platform team)
├── README.md
├── CODEOWNERS                            who must approve changes to each folder
├── docs/
│   ├── architecture/                     C4 diagrams, ADRs (decision records)
│   ├── runbooks/                         incident, rollback, key rotation, region failover
│   └── standards/                        naming, logging, prompt, tool and agent standards
├── infra/
│   ├── modules/                          network, kubernetes, redis, postgres, search, kms
│   ├── landing-zone/                     subscriptions, policies, private endpoints
│   └── envs/{dev,test,staging,prod-eu,prod-us}/
├── services/
│   ├── model-gateway/                    routing.yaml, quotas, cache, fallback
│   ├── guardrails/                       content, pii, injection, grounding checks
│   ├── tool-registry/                    catalogue API + approval workflow
│   ├── mcp-servers/
│   │   ├── erp/
│   │   ├── crm/
│   │   ├── hr/
│   │   └── documents/
│   ├── agent-runtime/                    graph execution, checkpoints, resume
│   ├── memory/                           short_term/, long_term/, write_rules/
│   ├── approvals/                        queue, notifier, decision log
│   ├── knowledge/                        ingestion/, classification/, indexes/
│   └── observability/                    otel collector config, dashboards, alerts
├── policy/                               policy as code
│   ├── data_access.rego                  who may read which classification
│   ├── tool_scopes.yaml                  which agent may call which tool
│   ├── risk_tiers.yaml                   tier -> approval rule
│   └── model_usage.yaml                  which data may go to which model
├── evaluation/
│   ├── golden_sets/{hr,finance,procurement}/
│   ├── red_team/                         injection, leakage, jailbreak, tool-abuse suites
│   ├── metrics/                          correctness, grounding, refusal, cost
│   └── gates/                            thresholds per risk tier
├── governance/
│   ├── model_cards/
│   ├── use_case_register/                one file per use case: owner, tier, data, controls
│   ├── dpia_templates/                   privacy impact assessments
│   └── audit_exports/
├── sdk/                                  libraries solution teams import
│   ├── python/northwind_ai/              client for gateway, tools, memory, approvals
│   └── contracts/                        OpenAPI + JSON schemas, versioned
├── pipelines/                            CI/CD definitions, release rings
└── tools/                                scaffolding: `new-solution`, `new-mcp-server`

procurement-agent/                        (a solution repo, owned by a solution team)
├── README.md
├── agents/
│   ├── supervisor.py
│   ├── supplier_agent.py
│   ├── contract_agent.py
│   └── policy_agent.py
├── graph.py                              nodes, edges, approval pauses
├── prompts/                              versioned, reviewed
├── tools.yaml                            tools requested from the registry, with scopes
├── eval/                                 solution golden set + red-team additions
├── tests/
└── deploy/                               manifests, release ring settings
```

6. The key artefacts in code and configuration

Model gateway routing (configuration, not code)

```
routes:
  - match: {task: classify, risk_tier: [0, 1]}
    model: small-fast             # cheap default
    fallback: [small-backup]
    cache_ttl_seconds: 3600
  - match: {task: reason, risk_tier: [2, 3]}
    model: strong-reasoning
    fallback: [strong-backup]
    require: [guardrails.input, guardrails.output, trace]
  - match: {data_class: restricted}
    model: self-hosted-private    # data never leaves the network
quotas:
  default: {requests_per_min: 600, monthly_budget_usd: 5000}
  teams:   {finance: {monthly_budget_usd: 20000}}
```

Policy as code: risk tiers and tool scopes

```
# policy/risk_tiers.yaml
tiers:
  0: {approval: none,        log: standard}
  1: {approval: none,        log: standard, sample_review: 0.05}
  2: {approval: one_human,   log: full,     evidence_required: true}
  3: {approval: two_humans,  log: full,     evidence_required: true, autonomy: never}

# policy/tool_scopes.yaml
agents:
  supplier_agent: {tools: [erp.read_supplier, documents.search], max_tier: 1}
  finance_agent:  {tools: [erp.read_ledger, erp.update_supplier], max_tier: 2}
```

A supervisor graph (outline of the shape, not a complete program)

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict):
    request: str; findings: dict; risk_tier: int; approved: bool | None; result: str | None

g = StateGraph(State)
g.add_node("supervisor", supervisor)        # decides which specialists to run
g.add_node("supplier",   supplier_agent)
g.add_node("policy",     policy_agent)
g.add_node("assess",     assess_risk)       # sets risk_tier from policy/risk_tiers.yaml
g.add_node("approval",   wait_for_human)    # pauses the run, resumes on decision
g.add_node("act",        execute_change)    # only reachable after approval for tier >= 2

g.set_entry_point("supervisor")
g.add_conditional_edges("supervisor", route_to_specialists)
g.add_edge("supplier", "assess"); g.add_edge("policy", "assess")
g.add_conditional_edges("assess", lambda s: "approval" if s["risk_tier"] >= 2 else "act")
g.add_conditional_edges("approval", lambda s: "act" if s["approved"] else END)
g.add_edge("act", END)
app = g.compile(checkpointer=durable_store)   # state saved at every step: resume, replay, audit
```

Memory write rule

```
def write_long_term(case, approved_by):
    assert approved_by, "long-term memory only accepts human-validated outcomes"
    case = mask_pii(summarise(case))            # store a summary, not the transcript
    store.upsert(case, metadata={"unit": case.unit, "tier": case.tier, "expires": today() +
RETENTION[case.tier]})
```

7. Memory in an enterprise agent system

Kind	Holds	Stored in	Rule
Short-term (working)	The current run: messages, tool outputs, hypotheses, plan	Run state with checkpoints (Redis or Postgres)	Cleared when the run closes; summarised when long
Episodic (long-term)	Past cases: what happened, what was decided, outcome	Vector store with metadata	Write only after human validation; summarise; expire by tier
Semantic (long-term)	Facts and documents: policies, product knowledge	Vector + keyword index	Versioned; superseded versions hidden; owner for each source
Procedural (long-term)	How to do things: playbooks, approved workflows	Versioned files and workflow definitions	Changed by pull request and review, never by the agent itself

8. Security, risk and compliance

Threat or duty	Control
Prompt injection through documents, emails, tool results	Treat all external text as data; scan at ingestion and at use; tool scopes cap the damage; red-team suite on every release
Data leakage across business units	Per-tenant indexes or enforced filters; classification labels; data-to-model rules in policy; leakage tests
Excessive agency (an agent does too much)	Least-privilege scopes per agent; tier-based approvals; write tools behind approval; rate limits on actions
Credential misuse	Workload identity, short-lived tokens, no keys in prompts or code, key rotation runbook
Model supply chain	Approved model list in the inventory; pinned versions; evaluation before upgrading
Regulation (privacy, sector rules, the EU AI Act risk categories)	Use-case register with risk tier; privacy impact assessment; documented human oversight; retention and deletion; audit export
Accountability	A named owner per use case; an AI council that approves tier 2 and 3; incident process that includes model failures

9. Reliability, scale and cost (FinOps)

- **Service levels:** define SLOs (for example 99.5% availability, answer p95 under 8 s) and an error budget; separate SLOs for the platform and for each solution.

- **Failure handling:** timeouts and retries with backoff on every call; circuit breakers on tools; model fallback in the gateway; degrade to "search only" or "hand to a human" instead of failing.
- **Resilience:** multi-region deployment, data replicated within residency rules, tested failover, and a manual mode for critical workflows.
- **Capacity:** autoscale on queue length and concurrent runs; reserve quota for critical solutions; protect with per-team rate limits.
- **Cost visibility:** every call tagged with team, solution and use case; budgets and alerts per team; monthly report showing cost per answer and per completed workflow.
- **Cost levers:** routing to small models, caching, shorter contexts, batching offline work, limiting agent steps, and removing unused solutions.

10. Delivery: the order to build it

Stage	Duration	Deliver	Do not do yet
0. Foundations	1 to 2 months	Landing zone, identity, network, CI/CD, model gateway (basic), tracing, use-case register, risk tiers agreed	Multi-agent workflows
1. First solutions	2 to 3 months	Two Level 2 assistants on the platform; guardrail service; evaluation service; knowledge ingestion with classification	Write actions
2. Tools and MCP	2 months	Tool and MCP registry; three enterprise systems exposed read-only; scopes enforced by policy	Autonomous actions
3. Agents and approvals	2 to 3 months	Agent runtime; memory service; approval service; first workflow agent at tier 1 to 2	Tier 3 automation
4. Scale and govern	ongoing	Multi-region, FinOps, red-team programme, audit exports, solution onboarding kit, platform SLOs	Large changes without evaluation

Teams and responsibilities

Group	Owns	Does not own
Platform team	Gateway, guardrails, registries, runtime, memory, observability, evaluation service, standards	Individual solution logic
Solution teams	Prompts, agents, workflows, solution golden sets, user experience	Shared infrastructure and policies
Data owners	Source quality, classification, retention	Model behaviour
Security and risk	Policies, red-team, approvals for tier 2 and 3, audits	Day-to-day releases
AI council	Use-case approval, risk appetite, exceptions	Engineering decisions

11. Common mistakes

- Starting with agents and autonomy instead of the platform basics: identity, gateway, evaluation, observability.
- Every team calling models and tools directly, so there is no single place to control cost, safety or audit.
- Memory that stores raw conversations or lets agents write anything they like.

- Approvals that exist on paper only: no evidence shown to the approver, or approvers who rubber-stamp.
- Treating evaluation as a one-time launch test instead of continuous.
- Choosing a framework before understanding the control flow, state and failure model you need.
- No exit plan for models or vendors.

12. Readiness checklist

- □ Every model call goes through the gateway and is tagged by team and use case
- □ Every tool is in the registry with an owner, schema, scope and risk class
- □ Risk tiers and approval rules exist as policy files and are enforced in code
- □ Tier 2 and 3 steps pause for a human and record the decision with evidence
- □ Long-term memory accepts only validated, masked, expiring entries
- □ Every release passes the evaluation gate and the red-team suite
- □ Traces connect user, agent, tool and model for each request
- □ Failover and rollback were tested, and SLOs are reported
- □ Use-case register, model inventory and audit export are current

13. How the three levels fit together

Question	Level 1: small	Level 2: medium	Level 3: enterprise
Who uses it	One team	One organisation	Many business units
Core pattern	Retrieve and answer	Retrieve, use tools, agent loop	Platform with supervised multi-agent workflows
Memory	None beyond the question	Session (short-term)	Short-term + episodic + semantic + procedural
Safety	Threshold and sources	Roles, guardrails, attack tests	Policy as code, risk tiers, approvals, red-team, audit
Quality	Manual 30 questions	Golden set in CI	Shared evaluation service and continuous monitoring
Operations	Logs	Tracing and dashboards	SLOs, FinOps, multi-region, incident process
Team	1 to 3	4 to 8	Platform team plus solution teams
Start here when	A pilot is enough	Real users and permissions	Many teams, regulation, actions with consequences

> **The rule that holds at every level**
> Start at the smallest level that meets the need, measure it, and move up only when a specific signal tells you to. Each level keeps the ideas of the one below and adds control around them.

Appendix A. A detailed sample deployment, with zones, services and ports

This appendix turns the reference architecture into one concrete, readable deployment for **Northwind** in a single region (West Europe, called `prod-eu`). Use it as a template for design reviews, firewall requests and network diagrams.

A.1 Network zones and address plan

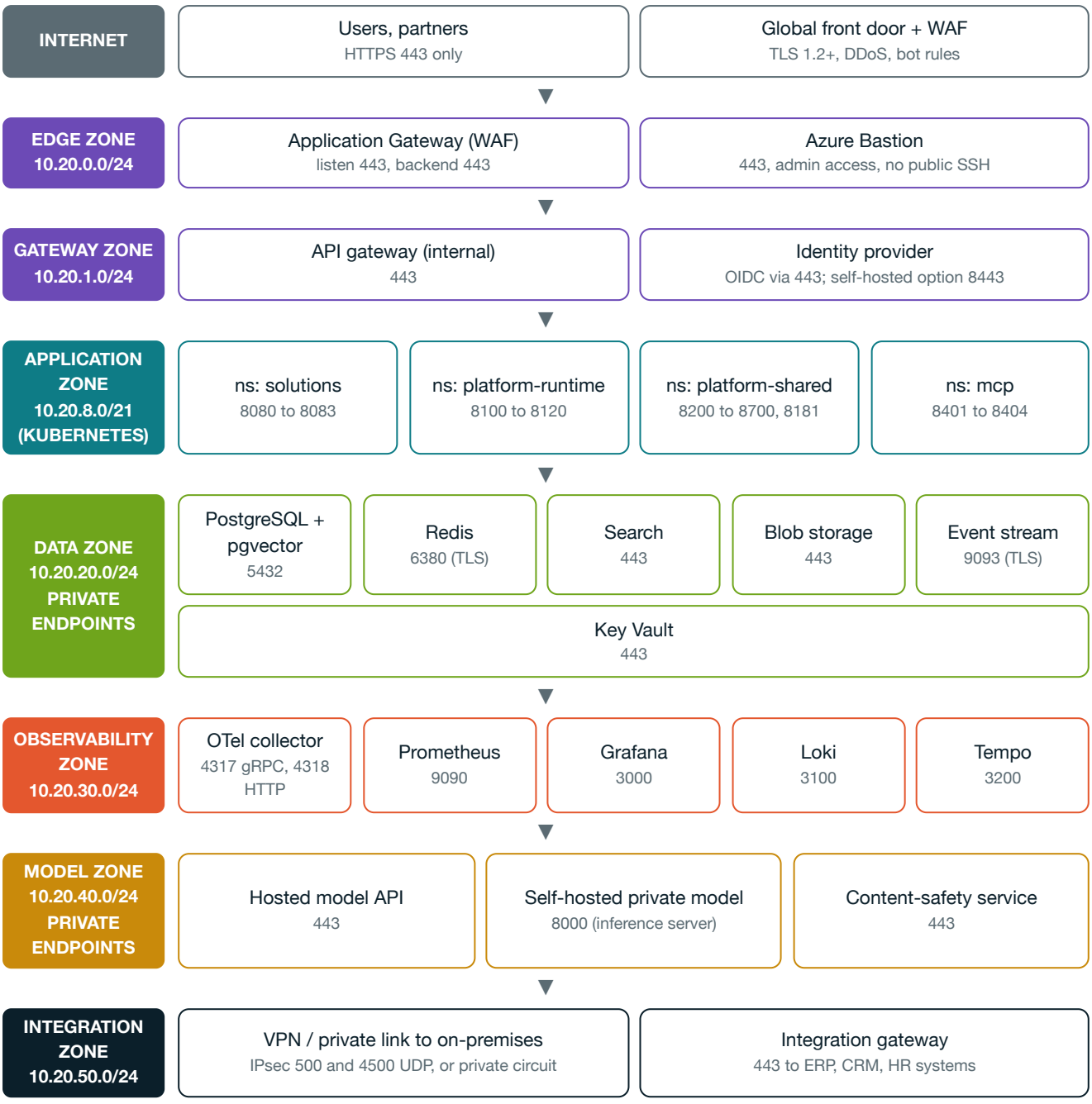


Figure A1. One region, one virtual network (10.20.0.0/16), eight zones. Nothing in the data, model or integration zones has a public address.

How to read the port numbers
Standard ports are the usual defaults of the product (HTTPS 443, PostgreSQL 5432, Redis TLS 6380, OpenTelemetry 4317 and 4318, Prometheus 9090, Grafana 3000, Kafka-compatible TLS 9093, Open Policy Agent 8181, MLflow 5000). **Assigned ports** (the 8xxx numbers for Northwind's own services) are chosen for this example so that each service has a distinct, memorable number. In a real project, use your organisation's port register, and remember that managed cloud services are reached on 443 through private endpoints.

A.2 Service and port catalogue

Zone / namespace	Service	Port	Protocol	Purpose	Talks to
Edge	Application Gateway (WAF)	443	HTTPS	Terminates TLS, WAF rules, routes by host and path	API gateway 443
Gateway	API gateway	443	HTTPS	Authentication check, quotas, request logging	solutions ingress 443
Gateway	Identity provider (OIDC)	443 (8443 if self-hosted)	HTTPS	Issues and validates tokens, roles and groups	directory, token validation by gateway
Application / solutions	web-app	8080	HTTP (inside mesh mTLS)	Chat and workflow user interface backend	hr-assistant, finance-assistant, procurement-agent
Application / solutions	hr-assistant	8081	HTTP	Knowledge assistant for HR and policy	agent-runtime 8100
Application / solutions	finance-assistant	8082	HTTP	Knowledge assistant for finance	agent-runtime 8100
Application / solutions	procurement-agent	8083	HTTP	Multi-agent procurement workflow	agent-runtime 8100
Application / platform-runtime	agent-runtime	8100 (HTTP), 8101 (gRPC)	HTTP, gRPC	Runs agent graphs, checkpoints state, resumes after approval	memory 8110, approvals 8120, model-gateway 8200, guardrails 8210, tool-registry 8300
Application / platform-runtime	memory-service	8110	HTTP	Short-term state and long-term memory API	Redis 6380, PostgreSQL 5432
Application / platform-runtime	approvals-service	8120	HTTP	Creates and resolves human approval tasks	PostgreSQL 5432, notifier (chat or email) 443

Application / platform-shared	model-gateway	8200	HTTP	Routing, quotas, cache, fallback for all model calls	hosted model 443, private model 8000, Redis 6380
Application / platform-shared	guardrails	8210	HTTP	Input and output checks, PII, injection, grounding	content-safety 443
Application / platform-shared	prompt-registry	8220	HTTP	Versioned prompts and approvals	PostgreSQL 5432
Application / platform-shared	tool-registry	8300	HTTP	Catalogue of tools, owners, scopes, schemas	PostgreSQL 5432, policy engine 8181
Application / platform-shared	knowledge-api	8500	HTTP	Search with role and tenant filters	PostgreSQL 5432, search 443
Application / platform-shared	evaluation-service	8600	HTTP	Runs golden sets and red-team suites	model-gateway 8200, MLflow 5000
Application / platform-shared	governance-registry	8700	HTTP	Model, tool and use-case inventory, audit exports	PostgreSQL 5432, Blob 443
Application / platform-shared	policy engine (Open Policy Agent)	8181	HTTP	Evaluates policy as code: data access, tool scopes, tiers	called by gateway, tool-registry, agent-runtime
Application / platform-shared	MLflow tracking	5000	HTTP	Experiment and evaluation results	PostgreSQL 5432, Blob 443
Application / mcp	mcp-erp	8401	HTTP (streamable)	MCP server for ERP read and approved write tools	integration gateway 443
Application / mcp	mcp-crm	8402	HTTP (streamable)	MCP server for CRM	integration gateway 443
Application / mcp	mcp-hr	8403	HTTP (streamable)	MCP server for HR system	integration gateway 443
Application / mcp	mcp-documents	8404	HTTP (streamable)	MCP server over document stores	Blob 443, knowledge-api 8500
Application / data-jobs	ingestion-worker	none inbound	Kafka protocol over TLS	Consumes new-document events, chunks, embeds, indexes	event stream 9093, Blob 443, PostgreSQL 5432
Data	PostgreSQL + pgvector	5432	PostgreSQL over TLS	Chunks, metadata, checkpoints, registries, audit tables	(accepts only from platform namespaces)

Data	Redis	6380	Redis over TLS	Session state, cache, rate-limit counters	(accepts only from memory, gateway)
Data	Event stream	9093	Kafka protocol over TLS	Document-change and audit events	ingestion, audit sink
Data	Blob storage and Key Vault	443	HTTPS	Source files, exports; secrets and keys	via private endpoints
Observability	OTel collector	4317 (gRPC), 4318 (HTTP)	OTLP	Receives traces, metrics and logs from every service	Tempo 3200, Prometheus 9090, Loki 3100
Observability	Prometheus / Grafana / Loki / Tempo	9090 / 3000 / 3100 / 3200	HTTP	Metrics store, dashboards, log store, trace store	Grafana reads the other three
Model	Hosted model API	443	HTTPS	Large language and embedding models	only from model-gateway
Model	Private inference server	8000	HTTP	Self-hosted model for restricted data	only from model-gateway
Mesh	Service-mesh sidecar	15001 out, 15006 in, 15021 health	TCP	Mutual TLS between pods, retries, timeouts	every pod

A.3 One request, hop by hop

A procurement buyer asks the agent to change a supplier's bank details. Follow the numbers in the table and in Figure A2.

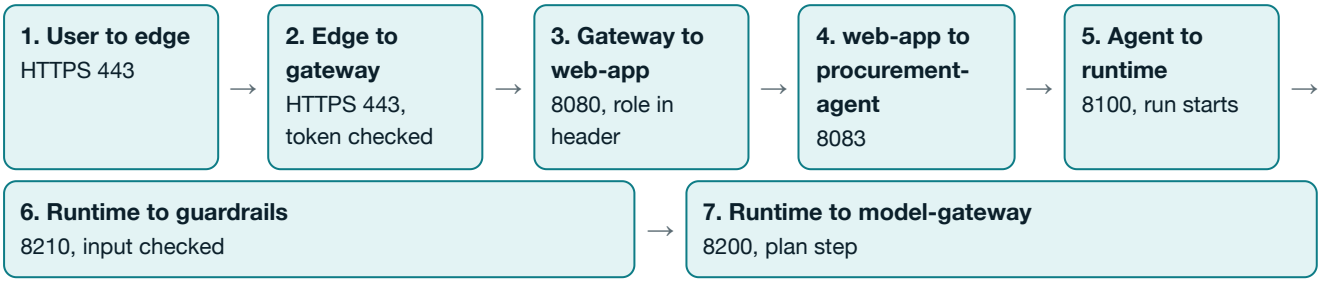


Figure A2 (part 1). From the browser to the first model call.

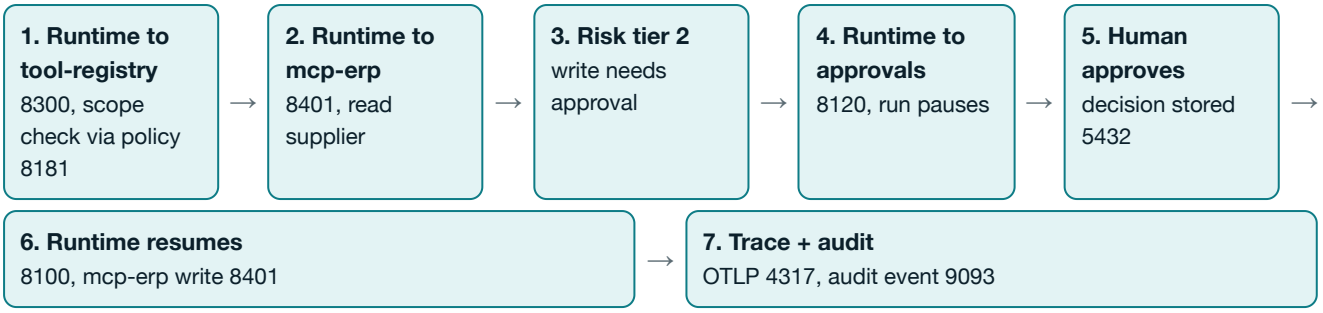


Figure A2 (part 2). Tool call, approval, resume, and the evidence trail.

#	From	To	Port	Authentication	What is recorded
1	Browser	Global front door and gateway	443	User signs in; token issued	Request id, client address (hashed)
2	Gateway	Identity provider (token check)	443	Signature and expiry; role claim	Subject, role, decision
3	web-app	procurement-agent	8083	Mesh mutual TLS + user token passed on	Trace id created
4	procurement-agent	agent-runtime	8100	Workload identity	Run id, solution id
5	agent-runtime	guardrails	8210	Workload identity	Check result, rule hits
6	agent-runtime	model-gateway	8200	Workload identity + team tag	Model, tokens, cost, cache hit
7	model-gateway	Hosted model API	443 (private endpoint)	Managed identity or key from Key Vault	Latency, status
8	agent-runtime	tool-registry and policy engine	8300, 8181	Workload identity	Tool, scope, allow or deny
9	agent-runtime	mcp-erp	8401	Short-lived token for the user and agent	Tool call, arguments (masked), result size
10	mcp-erp	Integration gateway	443	Service principal	System, operation
11	agent-runtime	approvals-service	8120	Workload identity	Task created, evidence attached
12	Approver	approvals-service	443 via gateway, 8120 inside	User sign-in with step-up	Who approved, when, what they saw
13	All services	OTel collector	4317 / 4318	Mesh mutual TLS	Spans linked by trace id

A.4 Firewall and network policy matrix

Default is **deny**. Only the rows below are allowed. Any other path is blocked and alerted.

Source	Destination	Port	Rule	Why

Internet	Edge (WAF)	443	Allow	Only public entry point
Edge	Gateway	443	Allow	WAF to API gateway
Gateway	Application / solutions	443 (internal load balancer)	Allow	Authenticated traffic only
solutions	platform-runtime	8100, 8101	Allow	Solutions use the runtime, nothing else
solutions	platform-shared, data, model	any	**Deny**	Solutions must not bypass the runtime and gateway
platform-runtime	platform-shared	8181, 8200, 8210, 8220, 8300, 8500	Allow	Runtime uses shared services
platform-runtime	mcp	8401 to 8404	Allow	Tool calls
platform-shared	Data	5432, 6380, 443, 9093	Allow	Shared services own data access
platform-shared (model-gateway only)	Model zone	443, 8000	Allow	Single exit to models
mcp	Integration zone	443	Allow	Only MCP servers talk to enterprise systems
Any pod	Observability	4317, 4318	Allow	Telemetry
Operations	Application	6443 (Kubernetes API) via Bastion	Allow, time-limited	Admin access with approval and recording
Any	Internet (egress)	any	**Deny**, except an allow-list of 443 destinations	Prevents data exfiltration and surprise downloads

A Kubernetes network policy that enforces one of those rows

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: solutions-egress, namespace: solutions}
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector: {matchLabels: {name: platform-runtime}}
      ports:
        - {protocol: TCP, port: 8100}
        - {protocol: TCP, port: 8101}
    - to:                                  # DNS
        - namespaceSelector: {matchLabels: {name: kube-system}}
      ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
```

A service definition with its assigned port

```
apiVersion: v1
kind: Service
metadata: {name: model-gateway, namespace: platform-shared}
spec:
  selector: {app: model-gateway}
  ports: [{name: http, port: 8200, targetPort: 8200}]
---
# in the pod: container listens on 8200; readiness probe GET /health on 8200
```

A.5 Configuration a solution team needs (names only, values come from the platform)

```
NW_GATEWAY_URL=http://model-gateway.platform-shared.svc:8200
NW_RUNTIME_URL=http://agent-runtime.platform-runtime.svc:8100
NW_TOOL_REGISTRY_URL=http://tool-registry.platform-shared.svc:8300
NW_MEMORY_URL=http://memory-service.platform-runtime.svc:8110
NW_APPROVALS_URL=http://approvals-service.platform-runtime.svc:8120
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector.observability.svc:4317
NW_SOLUTION_ID=procurement-agent        # used for cost tagging and policy
NW_RISK_TIER_MAX=2                      # highest tier this solution may reach
# no model keys, no database passwords: identity comes from the workload
```

A.6 The same system on a laptop (local port map)

Developers run a small version with Docker Compose so they can test without the cloud. The port numbers are kept the same where possible, so configuration does not change between laptop and cluster.

Container	Host port	Container port	Replaces in the cluster
postgres (with pgvector)	5432	5432	Managed PostgreSQL
redis	6379	6379	Managed Redis (6380 TLS)
model-gateway (stub or small model)	8200	8200	Model gateway
agent-runtime	8100	8100	Agent runtime
tool-registry	8300	8300	Tool registry
mcp-documents (local files)	8404	8404	MCP document server
policy engine	8181	8181	Policy engine
mlflow	5000	5000	MLflow tracking
otel-collector + grafana	4317, 3000	4317, 3000	Observability stack
your solution	8080	8080	web-app / solution service

Check yourself

Pick any arrow in Figure 1 of the main guide and answer: which zone is it in, which port does it use, which rule in A.4 allows it, how is the caller authenticated, and where is it recorded? If you cannot answer all five, the design is not finished.